BCA · ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



ATM AND UPI TRANSACTION SIMULATOR

A Python-based banking operations simulation

M. Amrutha Varshini · Mariyum · Harshith
BCA – Artificial Intelligence and Machine Learning · III Semester
REVA University

Made with GAMMA



PROJECT PURPOSE

A safe way to learn transaction processing

A menu-driven Python application simulates everyday banking operations without connecting to a real bank or payment network.

Access

Verify a PIN and manage account access

Operations

Check balance, withdraw, deposit, transfer via UPI, view a mini-statement, and change the PIN

Practice

Practice validation, control flow, data structures, and file handling

Made with GAMMA

The project turns banking rules into program logic



Access control

Secure PIN verification with a maximum of three attempts.



Transaction rules

Withdrawal limits, minimum-balance checks, and UPI transfer limits.



Cash logic

Cash denomination calculation using ₹500, ₹200, and ₹100 notes.



Stored records

Transaction history stored as structured records.



Python foundations

Python loops, conditions, functions, lists, tuples, and exception handling.

Made with GAMMA

THE CORE JOURNEY

Seven menu options cover the core journey



1 · Balance Enquiry



2 · Cash Withdrawal



3 · Deposit



4 · UPI Transfer



5 · Mini-Statement



6 · Change PIN



7 · Exit

3 PIN attempts

₹20,000 daily withdrawal limit

₹500 minimum balance

₹5,000 UPI transfer cap

Fraud alert after three consecutive large transfers

Made with GAMMA

TOOLS AND CONCEPTS

Python keeps the simulator lightweight and teachable

Developed and tested using Python 3.13 (Python 3.10+ compatible).

IMPORTED MODULES

os

CHECKS FOR AND MANAGES THE SAVED PIN FILE

getpass

HIDES PIN INPUT IN THE TERMINAL

Language and environment

EXECUTION ENVIRONMENT

- Jupyter Notebook
- Python-compatible terminal

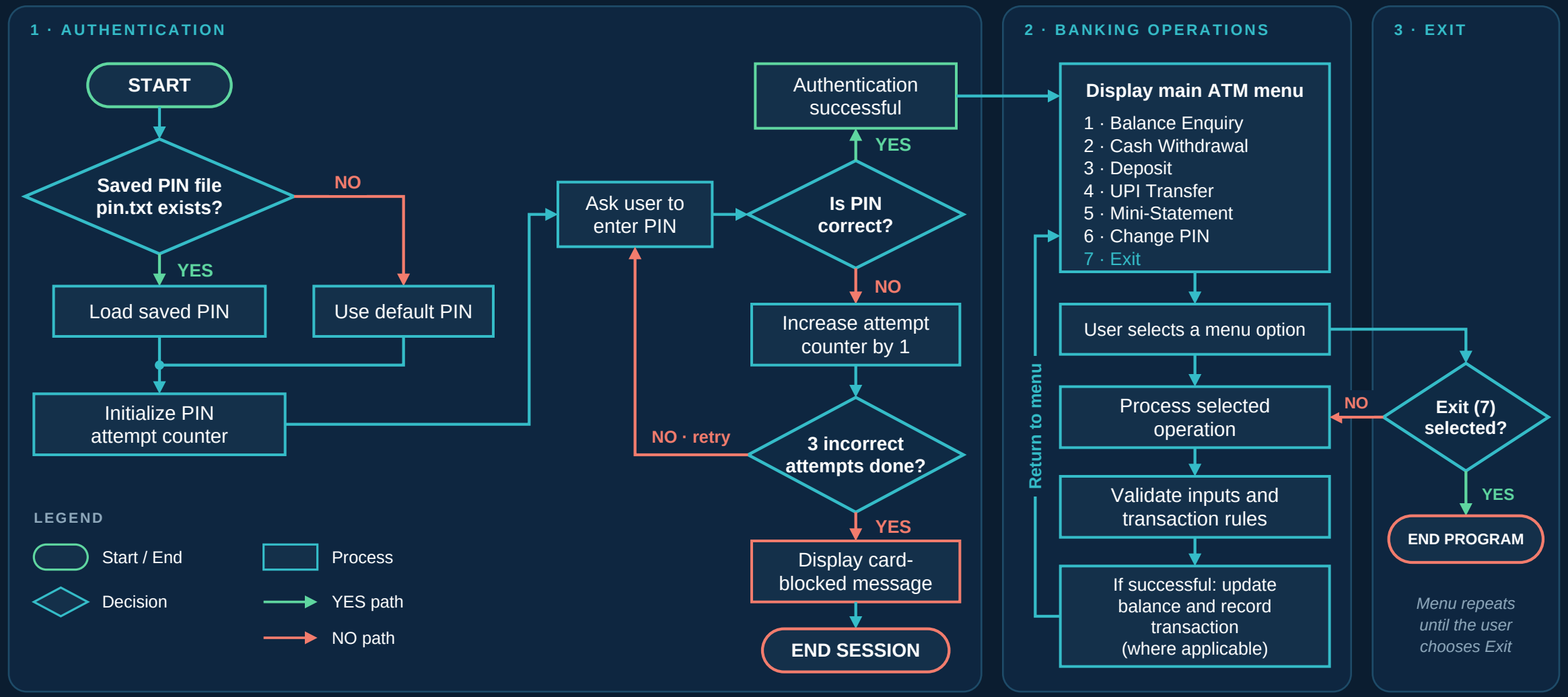
CORE CONCEPTS

Loops, functions, lists, tuples, slicing, break/continue, for-else, and try-except

Made with GAMMA

CONTROL FLOW

Authentication leads into a repeatable transaction loop



Made with GAMMA

IMPLEMENTATION PATTERNS

Five Python patterns carry the implementation

PIN verification

for attempt in range(1, 4), with break on success and for-else for the blocked-card path.

Denominations

Tuple (500, 200, 100) plus a loop and nested while.

Transactions

Tuples stored inside a list as (transaction_id, transaction_type, amount, balance).

Mini-statement

transactions[-5:] shows the latest five records.

Fraud detection

Counts consecutive successful large UPI transfers and alerts after three qualifying transfers.

Made with GAMMA

QUALITY AND SAFETY

Readable code, bounded inputs, basic educational security

Readable structure

Meaningful names, consistent indentation, comments, and a docstring in `record_transaction()`.

Bounded inputs

`try-except` handles non-numeric amounts; zero and negative values are rejected.

Validation

Menu choices, UPI ID format, limits, and minimum balance are validated.

Hidden terminal input

```
from getpass import getpass
entered_pin = getpass("Enter your 4-digit PIN: ")
```

PIN entry is hidden in the terminal; this is an educational measure, not banking-grade security.

Made with GAMMA

TESTING SCOPE

Testing covers the important failure paths

The separate testing report contains 15 test cases covering:

Authentication

Correct and incorrect PINs, three-attempt blocking, and PIN change.

ATM operations

Balance enquiry, valid and invalid withdrawals, minimum balance, and daily limit.

UPI and deposits

Deposits, UPI validation, transfer limit, and consecutive-transfer fraud alert.

Utility paths

Cash denomination calculation, mini-statement, invalid menu choices, and exit.

Made with GAMMA

TAKEAWAY

The simulator is intentionally a learning system

Limitations

PIN stored locally in a plain-text file. Session-only transaction history, daily withdrawal tracking resets, card blocking is not persistent, and basic UPI ID format validation.

Future

Database persistence, encrypted PIN storage, permanent history, GUI, multiple accounts, timestamps, and stronger monitoring.

Conclusion

The project connects Python fundamentals to a realistic transaction workflow.

THANK YOU



Made with GAMMA